



IP SERIES

Protocol Description

Date: 12TH JULY 2023
Document Revision: 1.02

TABLE OF CONTENTS

1. Protocol Description	3
2. Read device status	3
3. Configuration	4
3.1 Network	4
3.2 Users	5
3.3 General	6
4. Controlling device	6
4.1 Control PODs	7
4.2 Control Sequence	7

1. Protocol Description

IPSERIES used HTTP protocol to exchange status information and give access to control PODs. HTTP protocol description can be found here:

<http://www.w3.org/Protocols/rfc2616/rfc2616.html>

All commands and parameters are part of HTTP GET request. All status values are part of HTTP response

Each command will be described below.

2. Read device status

In order to read current status of device client should send HTTP GET request to **status.xml** file on the server. This file will be generated by server automatically and will include information about current state for:

- state (ON/OFF) of each POD
- SEQUENCE status (running/not running)
- Manual override switch state (ON/OFF)

The returned file will be formatted in following way:

```
<response>
<pod1>[POD1 STATE]</pod1>
<pod2>[POD2 STATE]</pod2>
<pod3>[POD3 STATE]</pod3>
<seq>[SEQUENCE STATE]</seq>
<mosws>[MANUAL OVERRIDE SWITCH STATE]</mosws>
</response>
```

POD STATE

Allowed values are 1 or 0.

0 – POD is turned off

1 – POD is turned on

SEQUENCE STATE

Allowed values are 1 or 0.

0 – SEQUENCE is not active

1 – SEQUENCE is active

MANUAL OVERRIDE SWITCH STATE

Allowed values are 1 or 0.

0 – Switch is turned off

1 – Switch is turned on

Example of request:

```
GET http://www.server.com/status.xml HTTP/1.1
Host: www.server.com
```

NOTE: Don't forget about two new line characters at the end of header

www.server.com - should be replaced with IP or domain name of real device

Example of XML response:

```
<response>
<pod1>1</pod1>
<pod2>0</pod2>
<pod3>1</pod3>
<seq>1</seq>
<mosws>0</mosws>
</response>
```

3. Configuration

Client can configure server using HTTP POST requests to the server. The server accepts three configuration requests: Network, Users, General.

Each configuration request described below.

3.1 Network

Network configuration includes:

- IP Address
- Network Mask
- Gateway IP
- DNS1 IP
- DNS2 IP
- TCP Port
- DHCP state

IP Address, Network Mask, Gateway IP, DNS1 IP, DNS2 IP should be formatted as IP address in IPv4 format:
xxx.xxx.xxx.xxx

TCP Port – it is integer number between 1 and 65535

DHCP State – integer number.

false – disable DHCP (will be used values from **IP Address, Network Mask, Gateway IP, DNS1 IP, DNS2 IP** fields)

true – enable DHCP (all values in **IP Address, Network Mask, Gateway IP, DNS1 IP, DNS2 IP** fields will be ignored)

Names of POST request variables

ip : IP Address
subnet : Network Mask
gw : Gateway IP
dns1 : DNS1 IP
dns2 : DNS2 IP
port : TCP Port
dhcp : DHCP state

In order to write new configuration to server client should compose new HTTP POST request to **net_config.cgi**. This request should contain POST variables with configuration values.

Example of request:

```
POST http://www.server.com/net_config.cgi HTTP/1.1
Host: www.server.com
Content-Length: 100
```

```
ip=192.168.0.10&subnet=255.255.255.0&gw=192.168.0.1&dns1=10.20.0.1&dns2=10.20.0.2&dhcp=false&port=80
```

IP Address	192.168.0.10
Network Mask	255.255.255.0
Gateway IP	192.168.0.1
DNS1 IP	10.20.0.1
DNS2 IP	20.20.0.2
TCP Port	80
DHCP state	false

Reading current Network configuration

Client cannot read network configuration directly.

3.2 Users

Users configuration includes:

- Admin username
- Admin password
- User username
- User password

All these are simple text strings.

Names of POST request variables

aname	: Admin username
apass	: Admin password
uname	: User username
upass	: User password

In order to write new configuration to server client should compose new HTTP POST request to **users_config.cgi**. This request should contain POST variables with configuration values.

Example of request:

```
POST http:// www.server.com/users_config.cgi HTTP/1.1
Host: www.server.com
```

```
aname=admin&apass=aaaa&uname=user1&upass=uuuu
```

Admin username	admin
Admin password	aaaa
User username	user1
User password	uuuu

Reading current Users configuration

Client cannot read users configuration directly.

3.3 General

General configuration includes:

- Serial port baudrate
- Power cycle status (enable /disable)
- PING address
- PING delay (in seconds)
- Number of failed pings after which system power cycle POD1

Serial port baudrate - number which present serial port valid baudrate: 300, 600, 1200, 2400, 4800, 9600, 19200, 38400, 57600, 115200

Power cycle status - state of Power Cycle POD1 function

0 – function is disabled

1 – function is enabled

PING address – IP address of domain name which should be pinged when function is enabled

PING delay (in seconds) – delay in seconds between consequences pings. Integer number greater then 0

Number of failed pings – integer number greater then 0. If system count failed pings equeal or greater then this value the power on POD1 will be cycled.

Names of POST request variables

baudrate	Serial port baudrate
pwrcyclestate	Power cycle status
pingaddr	PING address
pingperiod	PING delay (in seconds)
pingfails	Number of failed pings

In order to write new configuration to server client should compose new HTTP POST request to **users_config.cgi**. This request should contain POST variables with configuration values.

Example of request:

```
POST http:// www.server.com/common_config.cgi HTTP/1.1
Host: www.server.com
```

```
baudrate=300&pwrcyclestate=1&pingaddr=www.testdomain.com&pingperiod=10&pingfails=20
```

Serial port baudrate	300
Power cycle status	1 (enabled)
PING address	www.testdomain.com
PING delay (in seconds)	10
Number of failed pings	20

4. Controlling device

Client also can control remote device. It can switch POD satte (ON → OFF, OFF → ON) and start UP/DOWN sequences.

Each action described below.

4.1 Control PODs

Client can control 3 PODs on device: POD1, POD2, POD3. It can turn single POD on or off.

In order to control POD client should send HTTP GET request to one of the page:

pod1.cgi - control POD1
pod2.cgi - control POD2
pod3.cgi - control POD3

This request also should includes single query parameter with name **status**.

If parameter value is 0 – device should turn POD off

If parameter value is 1 – device should turn POD on

Example of request:

In order to turn POD1 off execute following request

```
GET http://www.server.com/pod1.cgi?status=0 HTTP/1.1
Host: www.server.com
```

In order to turn POD2 on execute following request

```
GET http://www.server.com/pod2.cgi?status=1 HTTP/1.1
Host: www.server.com
```

NOTE: Don't forget about two new line characters at the end of header

4.2 Control Sequence

Client can run UP/DOWN sequences on remote device.

In order to run sequence client should send HTTP GET request to the page **sequence.cgi**

The request should include two parameters: **status** and **delay**.

status – specify what type of sequence should be started

0 – DOWN sequence

1 – UP sequence

delay – number of seconds between POD actions in seconds. Should be integer number greater then 2.

Example of request:

Run UP sequence with delay 10 seconds

```
GET http://www.server.com/sequence.cgi?status=1&delay=10 HTTP/1.1
Host: www.server.com
```

Run DOWN sequence with delay 15 seconds

```
GET http://www.server.com/sequence.cgi?status=0&delay=15 HTTP/1.1
Host: www.server.com
```

NOTE: Don't forget about two new line characters at the end of header